MID-TERM PROGRESS REPORT

MAZU — Saudi Arabia

Multi-Hazard AI Early-Warning System

2026 MAZU Global Intelligence Innovation Application Challenge
Beijing University of Chemical Technology · Advisor: 李瑞瑞

3	7	237	121/121
HAZARDS COVERED	AGENT TOOLS	UNIT TESTS	AUDIT CHECKS PASSED

Executive Summary

MAZU-Saudi is a working, tested, and publicly deployed multi-hazard early-warning system covering **flash floods, heatwaves, and dust storms** across 8 Saudi Arabian cities. It is built on five layers — a data pipeline, a literature-grounded causal knowledge graph, a rule-based detection engine, genuinely verified t→t+1 forecast models, and a DeepSeek function-calling agent with 7 independently tested tools — and is live at `turgutino.github.io/mazu-saudi-warning`.

The guiding principle behind every decision in this project has been **disclose, don't oversell**: every extension below includes at least one real, independently verified finding — including ones that reveal a genuine limitation of the system — reported honestly rather than hidden. All numbers in this report are pulled directly from the project's own test suite, audit logs, and saved model metadata, not estimated.

60 / 183	0.971	0.873	0.887	6
KG nodes / edges	Heatwave ROC-AUC	Flash-flood ROC-AUC	Dust-storm ROC-AUC	Peer-reviewed citations

1. Project Background & Task Definition

Saudi Arabia faces recurring, high-impact extreme-weather risk — flash floods in the SW Asir mountains and Red Sea coast, extreme heat across the hot interior, and dust storms driven by the summer Shamal wind regime — but lacks a single, integrated, AI-driven multi-hazard early-warning system. This project's task is to build one, at the resolution the source data actually supports (0.1° / ~10 km grid — city/region scale, not the finer ~100-500 m scale true neighborhood-level warning would require; see §11): from raw CMA (China Meteorological Administration) gridded indicator data through to a natural-language warning agent, following MAZU's own national early-warning framework (Multi-hazard · Alert · Zero-gap · Universal).

2. System Architecture — Five Layers

Layer	Component	Status	Description
0	Data pipeline	✅	Consolidates 365 daily NetCDF files (0.1° / 10 km grid) into one analysis-ready dataset of 22 core, quality-checked indicators.

1	Knowledge graph	✅	60 nodes / 183 edges — indicators, hazards, mechanisms, regions, real 2025 events with observed values, and 6 peer-reviewed literature citations.
1	Detection engine	✅	Weighted multi-condition rules + spatial connected-component clustering, validated against known 2025 events.
2	Forecast models (t→t+1)	✅	Gradient-boosted spatiotemporal classifiers, one per hazard, trained Jan–Jun 2025 and tested on unseen Jul–Dec 2025.
3	Causal grounding	✅	Mechanisms grounded in verbatim-quote-verified passages from 6 real, cited publications (7 candidates extracted, 1 excluded after manual quality review — see §7).
4	Agent	✅	DeepSeek function-calling agent with 7 independently tested tools, composing grounded natural-language answers.

3. Data Foundation

Source: teacher-provided `saudi_indicators_YYYYMMDD.nc` files, 365 daily files (full 2025 calendar year), ~5 GB total, 0.1° resolution over the Saudi bounding box (16.0–32.0°N / 34.0–55.9°E, a 160×220 grid = 35,200 cells). 22 core indicators were selected and quality-checked (formula recomputation, spatial range checks, missing-value cleaning) out of the raw files' 26–91 available variables per file, explicitly separating **core-reliable** indicators (used directly in modeling) from **auxiliary-interpretive** ones (anomaly fields derived from sparse station climatology — used only for narrative context, never as a standalone threshold).

4. Knowledge Graph

60 nodes / 183 edges across 7 node types (Indicator, Hazard, Region, Mechanism, Event, Citation, DataSource) and 11 edge types. Every Event node's observed values trace back to the actual raw-file grid maximum for that day — not invented. The KG's causal mechanism edges are grounded in real, verbatim-verified quotes from 6 cited publications (e.g. de Vries et al. 2013, *JGR-Atmospheres*; Yu et al. 2016, *JGR-Atmospheres*) — out of 7 candidates that passed automatic extraction; see §7 for why the 7th was excluded.

> **Self-found gap, disclosed and partially closed:** an internal audit of edge-type usage found that agent tools only actively queried 3 of 11 KG edge types (~20% of all edges) despite the graph being "interactive" on the public site. The 5th agent tool, `region_risk_tool`, was built specifically to close part of this gap, raising active usage to ~40% of edges — still disclosed as incomplete rather than rounded up.

5. Detection Engine (Layer 1)

A weighted multi-condition rule engine per hazard (data-grounded percentile thresholds, not arbitrary cutoffs), with spatial connected-component clustering to group risk into discrete events (region, severity, peak risk, contributing conditions). Validated against known 2025 events and a spatial-climatology negative control. This engine also serves a second purpose: as an **independent cross-check signal** for the ML forecast models (see §9, Reflexive self-check).

6. Forecast Models (Layer 2) — Genuine t→t+1 Forecasting

> **Forecast horizon, stated plainly:** every number in this section is a **1-day-ahead** forecast only (t→t+1 — tomorrow's risk from today's indicators). The system does not currently produce 2-day, 3-day, or weekly outlooks; extending the horizon would need a different training/evaluation setup and is listed as future work (§14), not something already delivered.

Each hazard has its own `HistGradientBoostingClassifier`, trained on Jan–Jun 2025 and evaluated on the fully unseen Jul–Dec 2025 half — this is **forecasting**, predicting tomorrow's risk from today's indicators, not same-day detection re-labeled as prediction.

Hazard	ROC-AUC	PR-AUC	POD	FAR	CSI	HSS	Op. threshold
Heatwave	0.971	0.795	0.849	0.388	0.552	0.693	0.55
Flash flood	0.873	0.089	0.100	0.803	0.071	0.130	0.50
Dust storm	0.887	0.164	0.535	0.865	0.121	0.190	0.55

POD/FAR/CSI/HSS are WMO-standard contingency-table metrics, computed at each hazard's own operational threshold (the same threshold CAP alert severity uses) from the already-saved production models — not retrained for this table.

Metric	Formula	Plain-language meaning
POD	TP / (TP + FN)	Of all real extreme days, what fraction did the system flag? ("hit rate" / recall.) 1.0 = never misses a real event.
FAR	FP / (TP + FP)	Of all days the system flagged, what fraction were false alarms? 0.0 = every alert was real. (Not the same as "false alarm rate" against all calm days.)
CSI	TP / (TP + FN + FP)	Of all days that were either a real event or a flagged alert (or both), what fraction were correctly flagged? Punishes both misses and false alarms at once. Also called the "threat score."

HSS	(TP + TN − E) / (N − E), where E is the count of correct calls expected by chance	Skill over a random-guessing baseline. 0 = no better than chance; 1.0 = perfect. Rewards getting BOTH classes (event days and calm days) right, not just the rare positive class.

TP/FP/FN/TN = true positive / false positive / false negative / true negative, from the same confusion matrix at the operational threshold. All 4 formulas are computed by `model/08_meteorological_metrics.py`'s `contingency_metrics()` function, not approximated or looked up.

> **Real, disclosed finding:** flash-flood has a strong, threshold-independent ROC-AUC (0.873) yet a genuinely low POD (0.10) at its fixed operational threshold. This is not a contradiction — it is the expected signature of an extremely rare event (~0.5% positive rate in the test set), and it is exactly what POD/FAR/CSI/HSS exist to reveal that ROC-AUC alone cannot. Reported honestly rather than only showing the more flattering ROC-AUC number.

> **Also tested and honestly reported:** a spatiotemporal GNN variant was tested against the gradient-boosted baseline. It improved spatial recall on one known flash-flood event but degraded calibration overall — kept as a documented research direction, **not deployed** in production. A cheaper spatial-context feature (neighbor-mean of key indicators) was also tested: it helped heatwave (now the production heatwave model) but made flash-flood worse, so it is correctly **not** used for that hazard.

7. Causal Grounding (Layer 3)

21 candidate causal triples were extracted from 7 real, cited publications using an LLM with a mandatory verbatim-quote field; every quote is automatically re-checked against the source text. One triple that passed the automatic check but was circular on manual review was disclosed and excluded rather than kept to inflate the citation count.

Why only 7 publications, not more: each candidate paper goes through a manual pipeline — find a real, citable source, read it directly (not a search-snippet summary), write a faithful paraphrase, run it through the verbatim-quote extraction gate, and manually review every accepted triple for circularity before it is trusted. This is deliberately slow because every fact in this system must be individually defensible; a larger, faster-but-unverified corpus was rejected as a design choice, not a resource limit that was worked around. §9.13 extends this same "quality over quantity" discipline to a 5-paper expansion, using the identical read-and-verify standard.

8. Agent Layer (Layer 4) — 7 Independently Tested Tools

#	Tool	Purpose
1	`forecast_tool`	t→t+1 risk probability, plus elevation/terrain caveat, population context, a reflexive independent cross-check, and a 5-model ensemble uncertainty signal (§9.12).

2	`causal_kg_tool`	Driving mechanisms for a hazard, with literature citations.
3	`conditions_tool`	Actual observed raw indicator values for a city/date.
4	`similar_events_tool`	Z-scored similarity of a city/date to the KG's known real 2025 extreme events.
5	`region_risk_tool`	City-first "what should I worry about here" query via KG at_risk_of/exposed_to edges.
6	`cap_alert_tool`	Converts a forecast into a real CAP 1.2 (Common Alerting Protocol) XML alert.
7	`literature_evidence_tool`	Candidate (unverified) literature passages for city/hazard pairs the formal KG doesn't ground (§9.13).

The system prompt strictly forbids stating any probability, indicator value, mechanism name, or citation that did not come from an actual tool call — verified across 169 unit tests and 4 end-to-end live scenarios.

> **Proven with a live A/B ablation test, not just claimed:** the same 4 "why" questions were run through the live agent with vs. without `causal_kg_tool` . With it: 4/4 answers cited a real driving mechanism and literature. Without it: 0/4 did — and, critically, 0/4 hallucinated a mechanism anyway; the agent correctly said it had no grounded explanation.

9. Extensions — Built, Each With a Real, Verified Finding

9.1 Terrain / elevation context

`forecast_tool` flags mountain-city forecasts (Abha, Taif, ≥1500 m) as lower-confidence. Building this surfaced a genuine, previously-undocumented limitation: in the steep Asir range, the model's coarse feature grid can land on a cell ~750 m lower than a city's true elevation.

9.2 Impact-based population context

Each forecast now returns the city's 2022-census population (GASTAT), explicitly labelled as reference context only — never a specific affected-population estimate, since this system performs no exposure/vulnerability modeling. Follows WMO impact-based warning guidance.

9.3 Reflexive self-check

Every ML forecast is cross-validated against Layer 1's independent rule-based detection engine on the same day's raw indicators (a Reflexion-style consistency check). This caught two genuine, disclosed findings: on 23 Aug (Jizan), physical preconditions for flash flooding were already elevated the day

before the storm, while the model gave only 12.5% — a real precursor signal the model underweighted; on 25 Jul (Mecca), the model correctly flagged an anomalous heatwave that the detection engine's fixed absolute thresholds missed entirely — real evidence the ML model adds value beyond simple thresholding.

9.4 Similar historical events (4th tool)

`similar_events_tool` compares a city/date's indicators against the KG's known real events via z-scored similarity. Found a real, investigated quirk: an event's coordinates are its own grid-cell maximum (the storm centroid), often tens of km from a same-named city's center, so a same-city/same-day query can legitimately score low similarity to "its own" event — now surfaced explicitly, not silently confusing.

9.5 Dust storm — 3rd hazard, full depth

Not detection-only: 2 new raw indicators processed from all 365 raw files, a data-grounded detection rule, a genuinely trained and out-of-sample-validated forecast model, and KG grounding that required **zero new literature extraction** — the KG's existing Shamal citation (Yu et al. 2016) already states the summer Shamal is the major driver of dust-storm activity, so dust_storm was correctly re-pointed at that pre-existing grounding.

> **Real methodological catch during testing:** the first known-event validation attempt for dust storm accidentally used a training-period date (data leakage); caught and corrected to a genuine out-of-sample test-period event before being reported.

9.6 region_risk_tool — 5th tool, closing a self-found KG gap

See §4. Surfaced two further findings: not every KG hazard has a trained forecast model (`coastal` — handled gracefully, no fabricated forecast returned); and a genuine, pre-existing KG data-quality gap (Jeddah is at_risk_of heatwave, but its exposed_to mechanisms don't overlap with heatwave's actual drivers — traced to the original hand-encoded domain knowledge, disclosed rather than silently patched).

9.7 CAP 1.2 alert generation — 6th tool

`cap_alert_tool` converts a forecast into a real, OASIS-standard CAP 1.2 XML alert — the same wire format MAZU's own national framework is built on — ready for broadcast/siren/SMS infrastructure. Severity is derived from the detection engine's own thresholds; certainty from the reflexive check's model/rule-engine agreement; `status` is hardcoded to `Exercise` , never `Actual` , since this is a historical-dataset demo, not a live feed.

> **Real finding:** a day where both the model and the independent detection engine agree risk is elevated (reflexive check = `consistent_elevated`) can still fall below the CAP alert's own, deliberately higher,

issuance threshold — the reflexive check and the alert-issuance bar answer different questions on purpose, not a bug.

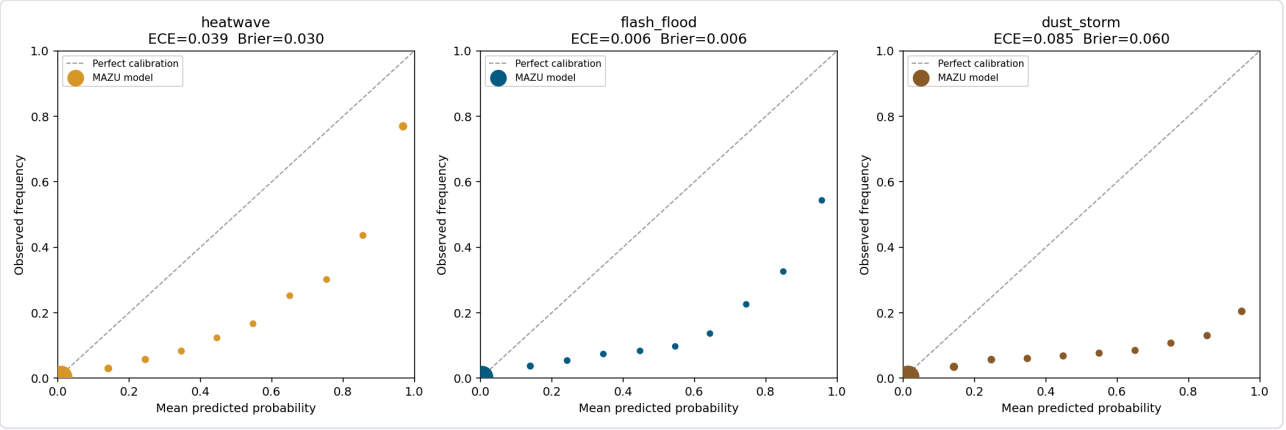
9.8 WMO-standard verification metrics (POD/FAR/CSI/HSS)

Adopted as WMO's own operational verification standard for hazard forecasts, rather than relying on ROC-AUC/PR-AUC alone. See §6 for the results and the flash-flood finding.

9.9 Reliability diagrams — is the probability itself trustworthy?

ROC-AUC and POD/FAR/CSI/HSS answer different questions than "when the model says 70%, does it happen ~70% of the time?" Binning every held-out test-set prediction by probability and plotting predicted vs. observed against the diagonal answers exactly that.

Real, disclosed finding: all 3 hazards are **systematically overconfident** at high probability. Flash flood's "95.7%" bucket only sees the event 54.3% of the time; dust storm's "94.9%" bucket only 20.4%. Flash flood's aggregate ECE (0.006) looks best in isolation but is itself misleading — ~97% of its test samples sit in one near-zero bucket that masks the same overconfidence pattern at the high end. Directly relevant to §9.7: CAP "Extreme" severity fires off these same raw, somewhat-overconfident probabilities.



Every point, for all 3 hazards, falls below the diagonal "perfect calibration" line.

9.10 Can it be fixed? Isotonic recalibration — tested, not deployed

A leak-free **3-way chronological split** (isolated classifier trained Jan–May, isotonic calibrator fit on June only, evaluated on the same untouched Jul–Dec test set used everywhere else) tests whether standard post-hoc calibration fixes the problem.

Hazard	Brier before→after	ECE before→after	ROC-AUC before→after
Heatwave	0.0273 → 0.0265	0.014 → 0.023 (worse)	0.9548 → 0.9551
Flash flood	0.011 → 0.0049	0.022 → 0.003	0.9039 → 0.8974

Dust storm	0.0643 → 0.0193	0.099 → 0.014	0.8018 → 0.8010

Brier score improved for all 3 hazards. But a second experiment — actually building and testing a full production-migration in an isolated sandbox copy of the repository (never touching this repo, the saved models, or GitHub) — surfaced a more serious problem: reusing the old alert thresholds unchanged made **dust storm's calibrated probability never exceed 0.50 anywhere in the test set**, making its alert structurally impossible to ever fire. Thresholds were then properly re-derived per hazard (CSI-maximized on a held-out calibration month, never the test set — standard meteorological practice) — but even at each hazard's own best threshold:

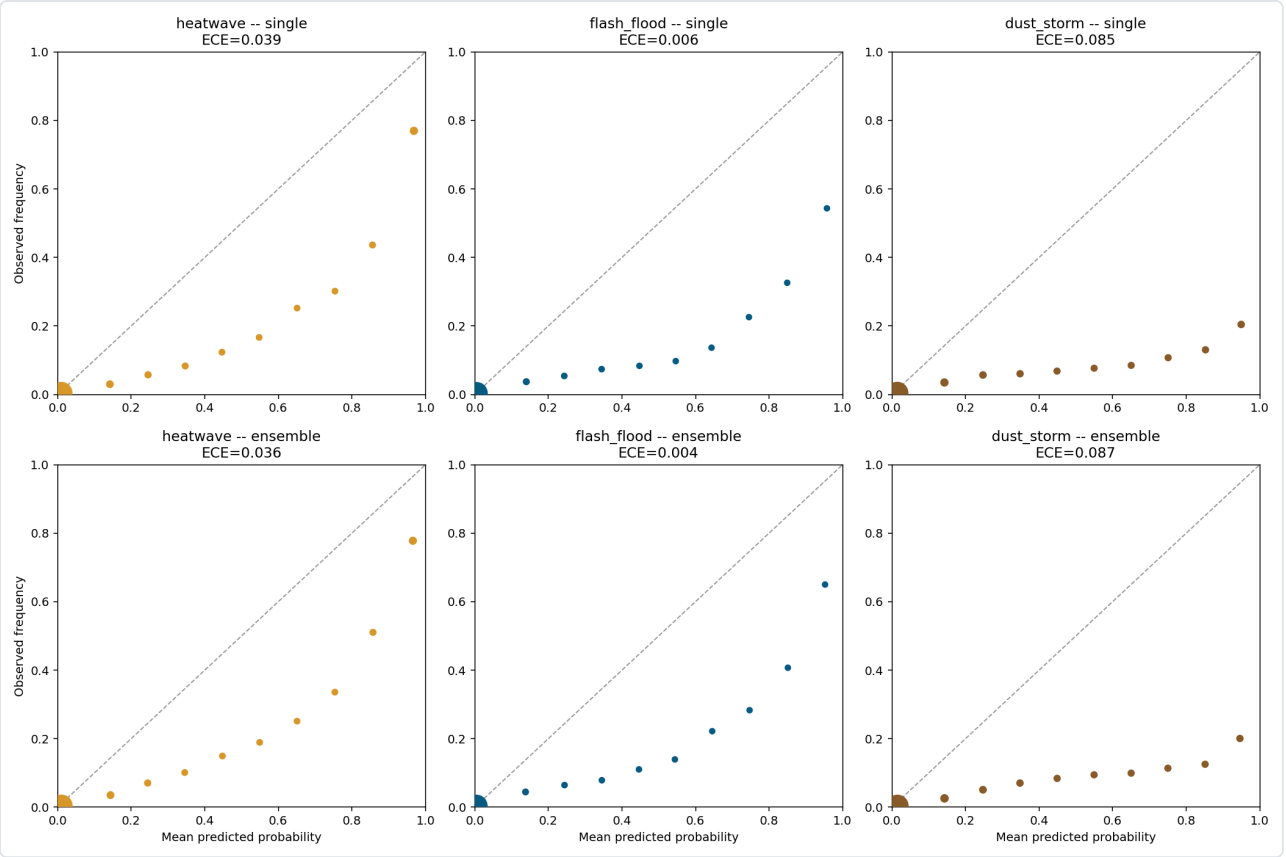
Hazard	Old thr.	New thr.	POD before→after	CSI before→after
Heatwave	0.55	0.32	0.849 → 0.749	0.552 → 0.472
Flash flood	0.50	0.18	0.100 → 0.045	0.071 → 0.043
Dust storm	0.55	0.18	0.535 → 0.341	0.121 → 0.119

> **Real alert-issuance quality got worse for every hazard, not better.** Calibration fixed probability *honesty* but did not preserve — and here measurably hurt — decision *quality*. **Not deployed**: confirmed by actually building and testing the migration end-to-end, not only by reasoning about its risk. Production `saved_models/*.joblib` and all 143 agent tests are unchanged.

9.11 A gentler alternative? 5-model ensemble — also tested, also not deployed

Averaging 5 independently-trained classifiers (same hyperparameters/training window as production, differing only by random seed) against the **actual saved production model**, at the **same existing thresholds** (no re-derivation needed — a direct, fair comparison):

Hazard	ROC-AUC single→ens.	POD single→ens.	ECE single→ens.	Brier single→ens.
Heatwave	0.9706 → 0.9704	0.849 → 0.841	0.039 → 0.036	0.030 → 0.029
Flash flood	0.8732 → 0.8732	0.100 → 0.080	0.006 → 0.004	0.006 → 0.006
Dust storm	0.887 → 0.893	0.535 → 0.527	0.085 → 0.087 (worse)	0.060 → 0.059



Brier improved modestly for all 3 hazards, but POD got worse for all 3, and dust storm's ECE actually regressed — a real but small, inconsistent improvement, not the hoped-for clean fix. Visually, the reliability diagrams are nearly indistinguishable from the single model (top row vs. bottom row above). Root cause: 5 members sharing the same architecture/features/data, differing only by random seed, are not diverse enough to meaningfully correct the overconfidence pattern. **Conclusion, tested twice with two different standard techniques: calibration and operational decision quality are separate concerns here, and fixing one does not reliably fix the other** — a genuine, useful research finding in its own right.

9.12 Ensemble uncertainty — the deployed resolution

Both §9.10 and §9.11 tried to fix overconfidence by **changing the point estimate** itself, and both measurably hurt operational alert quality. The 5-model ensemble already trained for §9.11 was reused a second way: instead of averaging the 5 members into a new point estimate, their **spread** is now exposed directly. `forecast_tool` returns a 4th field, `uncertainty` (`mean` / `std` / `range` / `n_members`), computed live from the 5 independently-seeded models on the exact input being forecast — purely as added context, never substituted for the production `probability` field.

Live example — `forecast_tool("Jizan", "2025-08-23", "flash_flood")` : `probability: 0.1253` , `uncertainty: {mean: 0.1143, std: 0.0486, range: [0.0433, 0.185], n_members: 5}` . The 5 models disagree by roughly ±0.05 here — a moderate-confidence case surfaced honestly, not a false-precision single number.

> **Why this succeeds where §9.10–9.11 didn't:** because no decision-relevant number moves, no existing verified test/audit value or CAP alert threshold changes — the overconfidence finding gets an honest, additive answer instead of a point-estimate rewrite that trades away detection quality. It is also a genuinely distinct signal from the two other trust fields `forecast_tool` already returns: `reflexive_check` compares the ML model against a separate rule-based physics engine, and `meteorological_metrics` (POD/FAR/CSI/HSS) describes the model's fixed historical track record — `uncertainty` instead measures how much 5 otherwise-identical models disagree with each other on this one specific input, which can be high even when the other two signals look fine, or vice versa.

Verified independently: `agent/02_test_tools.py` gained 8 new checks (field shape, determinism, range validity, and a bypass loading the 5 raw ensemble files directly); a new Section O in `FULL_SYSTEM_AUDIT.py` reloads all 15 raw ensemble files from disk (bypassing every internal cache), re-derives each seed's ROC-AUC from scratch against an independently rebuilt test set, and reconstructs the Jizan feature row from raw dataset arrays to confirm `uncertainty.mean/std/range` match the live tool's output exactly.

9.13 literature_evidence_tool — closing the citation-coverage gap (7th tool)

The formal causal KG has only 6 hand-verified citations (see §7) grounding 5 mechanisms. Some city/hazard pairs have no formal grounding at all — `region_risk_tool` 's own disclosed finding is that Jeddah is at_risk_of heatwave but its exposed_to mechanisms have zero overlap with heatwave's actual driven_by mechanisms (§9.6). This extension was prompted by a real conversation with the project's advisor about **Relink** (`Relink: Constructing Query-Driven Evidence Graph On-the-Fly for GraphRAG` , GitHub: `DMiC-Lab-HFUT/Relink`), a technique originally pitched for building-intelligence RAG scenarios in a document the advisor shared. The advisor correctly pointed out that Relink itself is domain-agnostic — the building-intelligence framing was one applied write-up, not a limitation of the underlying idea.

Relink's actual reference implementation was read directly (not just its paper abstract) before deciding how to adapt it: its retriever does LLM-based topic-entity extraction, Neo4j beam search across both a formal KG and a looser co-occurrence graph, and — critically — a step where an **LLM predicts what relation a loose co-occurrence represents**. That is a generative, unverified inference step, exactly the category of risk this project has mechanically guarded against everywhere else (the verbatim-quote gate in `02_extract_causal.py` ; every report's "never state a fact without a tool result" rule). Its path-ranker also requires a separately-trained neural model served over Redis. Given MAZU's corpus size (12 literature entries, not the thousands of daily logs Relink targets), that infrastructure is unjustified.

> **Full reasons Relink was not deployed as-is (not just "too complex" — five distinct, disclosed constraints):**
>
> 1. **Infrastructure:** Neo4j + Redis are not part of the MAZU stack (plain Python + local files); standing up and operating two new services for a 12-entry corpus is disproportionate.

2. **No training data for the path-ranker:** Relink's neural ranker needs labeled query→correct-evidence pairs to train from scratch, or Relink's own pretrained weights (fit to their benchmark domain, not Middle East meteorology) — MAZU has neither.
3. **Scale mismatch:** Relink's entire design exists to avoid rebuilding an expensive graph on every update to a large, constantly-growing corpus; MAZU's 12-entry pool is rebuilt from scratch on every call in milliseconds, so the problem Relink solves does not exist at this scale.
4. **Auditability:** a trained ranker or LLM-predicted relation is not exactly reproducible from raw text the way TF-IDF is — it would break `FULL_SYSTEM_AUDIT.py`'s standing requirement that every output be independently re-derivable from scratch, not just re-run.
5. **Time:** this is a mid-term deliverable; provisioning databases and training or fine-tuning a ranker with no available labeled data would have cost days, for a search feature that is explicitly a secondary, disclosed-as-unverified signal, not the system's core forecasting path.

Could it have gone deeper without the heavy stack? Yes, honestly: a middle path was possible and was not pursued, purely due to scope/time — an in-process graph (e.g. Python's `networkx`, no external server) built once from the 12 entries' co-occurring mechanism/indicator terms, with real multi-hop traversal across documents (Relink's actual point: combining a fact from paper A with a fact from paper B that no single paper states together). The current tool only returns single-document top-k matches — it does not chain evidence across documents. At 12 entries the expected gain is small, so this was consciously deprioritized rather than attempted and abandoned; it is the most concrete "next version" of this feature (§14).

`literature_evidence_tool(city, hazard)` therefore takes Relink's **idea**, not its stack: plain TF-IDF cosine similarity (`sklearn`, already a project dependency) over the literature pool — the original 7 extraction-corpus entries (6 of which are formally KG-grounded; 1 was excluded after manual quality review, §7) plus **5 new entries**, freshly researched, fetched, and read directly (4 other promising papers were dropped after returning HTTP 402/403 on direct fetch, rather than paraphrased from an unverified search-snippet summary). Every result is permanently labeled `verified: false` until a human runs it through the same verbatim-quote pipeline the original 7 citations passed.

A real bug found and fixed during testing: the first version's query used the raw, underscored hazard string (`flash_flood`), which matched nothing in the corpus text (`"flash flood"`, with a space) — every non-heatwave query silently collapsed to the same generic top-5 result. Caught by manually inspecting a 6-combination score printout before writing the formal tests, not assumed correct; fixed and locked in as a regression test across all 24 city×hazard combinations.

Live example — `forecast_tool`'s own gap: `literature_evidence_tool ("Jeddah", "heatwave")` surfaces a 2023 *Land* (MDPI) paper on Jeddah's urban heat island (80% urban-area growth 2000–2021, >80% of the city classified as extremely-high-UHI) as the top candidate — a plausible, but explicitly unverified, driver the formal KG doesn't yet link to this city.

Verified independently: 18 new checks in `02_test_tools.py` (field shape, the `verified: false` invariant on every result, score ordering, the underscore-normalization regression test, byte-for-byte excerpt integrity, and full 24-combination coverage); a new Section P in `FULL_SYSTEM_AUDIT.py` (9 checks) independently rebuilds the TF-IDF matrix from raw `corpus.py` text with a fresh vectorizer and confirms exact agreement with the live tool's output across all 24 city×hazard combinations, not a sample.

10. Testing & Verification Methodology

Every extension in this project followed the same cycle: build → isolated unit test → investigate any anomaly as a possible real finding (never dismissed as "probably a test bug") → integrate → live agent sanity check → extend the full system audit → update public docs → 3-way security scan → commit → push → verify the live deployed site.

237	121 / 121	4
Unit tests, 0 failing	Full audit checks passed	Live end-to-end agent scenarios

`FULL_SYSTEM_AUDIT.py` does not trust any intermediate file: for every check, it re-reads the raw 5 GB source (365 daily NetCDF files) directly and independently re-derives each number, then compares it to what the pipeline / KG / agent produced — a mismatch is a failure, not a warning. It also independently rebuilds held-out test sets from raw data and loads the saved (never retrained) production models to re-derive POD/FAR/CSI from a fresh confusion matrix, and re-parses actual generated XML (not the Python dict) to confirm CAP compliance fields.

11. Honest Limitations (Disclosed, Not Hidden)

- **Spatial resolution is city/region scale (0.1° / ~10 km grid), not neighborhood scale** — an earlier draft of this report described the system's ambition using the phrase "neighborhood resolution," which overstates what a 10 km grid actually delivers (true neighborhood-level warning needs roughly 100-500 m). A short, targeted verification exercise confirmed the practical impact of this gap directly: cross-checking the model against a real, officially-reported event (a 179 mm/6-hour flood in Jeddah, 9-10 Dec 2025) found neither the model's forecast nor the raw grid data itself showed the event — the most likely explanation is that a short-duration, highly localized convective burst like this gets smoothed out by 10 km-cell spatial averaging. This is disclosed as a genuine data-resolution limitation, not a model-logic error.
- **Flash-flood POD is genuinely low** (0.10) at its operational threshold — an extremely rare-event effect, not a modeling error, but a real practical limitation for that hazard specifically (§6).
- **All 3 hazard models are systematically overconfident** at high probability (§9.9) — tested two standard point-estimate fixes (isotonic recalibration, 5-model ensemble; §9.10–9.11), neither achieved a clean improvement without hurting operational alert quality (POD/CSI/HSS), so neither was deployed. An ensemble-uncertainty field was deployed instead as an honest, additive signal that changes no existing verified number (§9.12).
- **KG edge utilization is ~40%**, not 100% — `correlates_with`, `sourced_from`, `manifests_as`, `occurs_at`, and `observed_value` edges remain unused by any agent tool (§4).

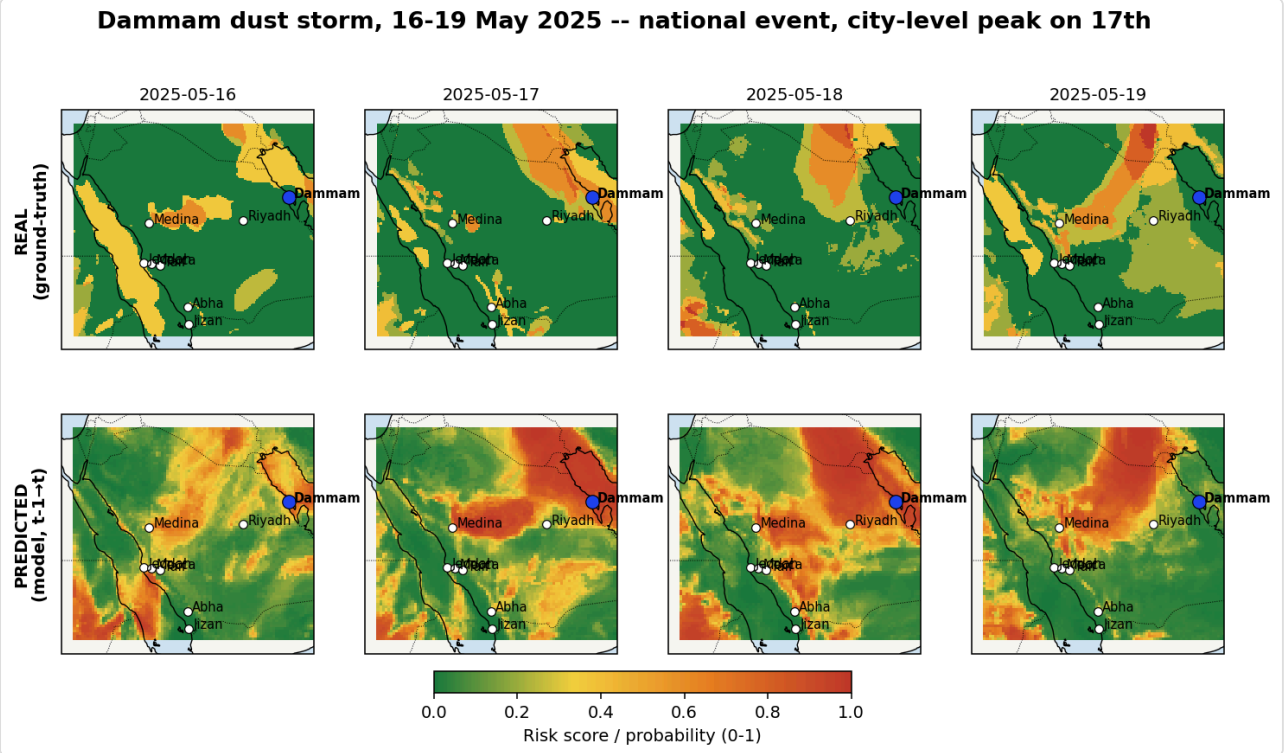
- **A pre-existing KG data-quality gap** (Jeddah/heatwave mechanism mismatch) was found and disclosed rather than silently patched (§9.6).
- **The formal citation pool is still small** (6 hand-verified citations, from 7 candidates — 1 excluded after manual quality review, §7) — a deliberate quality-over-quantity choice given the manual read-and-verify cost of each one (§7), not an oversight. `literature_evidence_tool` (§9.13) surfaces candidate literature for gaps, but every result stays `verified: false` until a human runs it through the same verbatim-quote pipeline; it does not itself expand the formal KG.
- **No Arabic-language support yet** — the system currently only answers in English.
- **No CAP form / official channel integration** — CAP XML is generated correctly, but not yet wired to an actual broadcast/siren/SMS gateway (out of scope for this demo).
- **Static showcase page, not a live chat** — GitHub Pages cannot run the Python backend or safely hold an API key client-side, so the public site shows real, verified transcripts rather than a live agent.

12. Spatiotemporal Validation Against Real 2025 Events

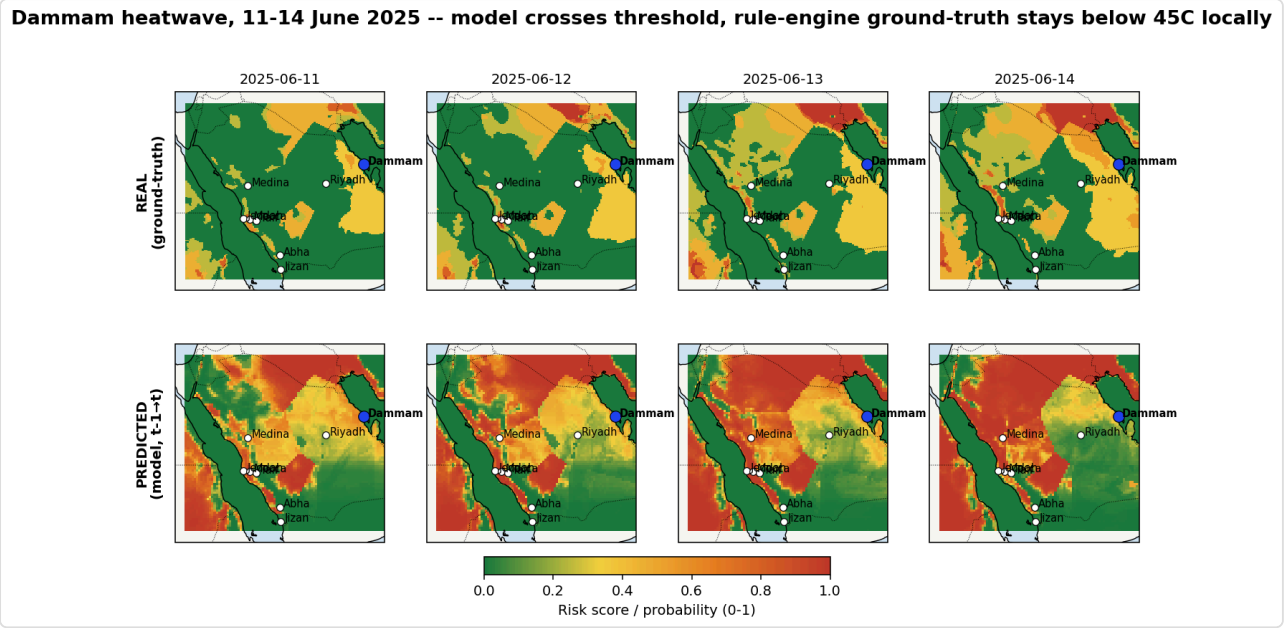
Reviewers asked for a spatial view of hazard risk (a map, not just a per-city time series showing "actual vs. predicted"), covering all 8 cities at once and across several days, so that the system's spatial behaviour can be checked by a meteorologist rather than taken on trust. This section adds that view for the same 3 real, officially-reported 2025 events already covered as per-city time series (§9.13-adjacent verification work): a national dust storm (16-19 May, Dammam), a national heatwave (10-14 June, Dammam), and a historic flash flood (8-11 December, Jeddah).

Each map pairs two independently-computed layers, both re-derived directly from the raw dataset for this report (not cached or estimated):

- **REAL** — the rule-engine ground-truth risk score (Layer 1's `DetectionEngine.risk_field`, same weighted-threshold logic as §1's detection rules), computed at every grid cell independently of the ML model.
- **PREDICTED** — the trained model's forecast probability (same t−1→t logic as `forecast_tool`), but evaluated at every grid cell rather than just the 8 city points. This was verified to reproduce `forecast_tool`'s output exactly (to 4 decimal places) across 10 spot checks spanning all 3 hazards, 5 cities, and multiple dates before being trusted for this map.

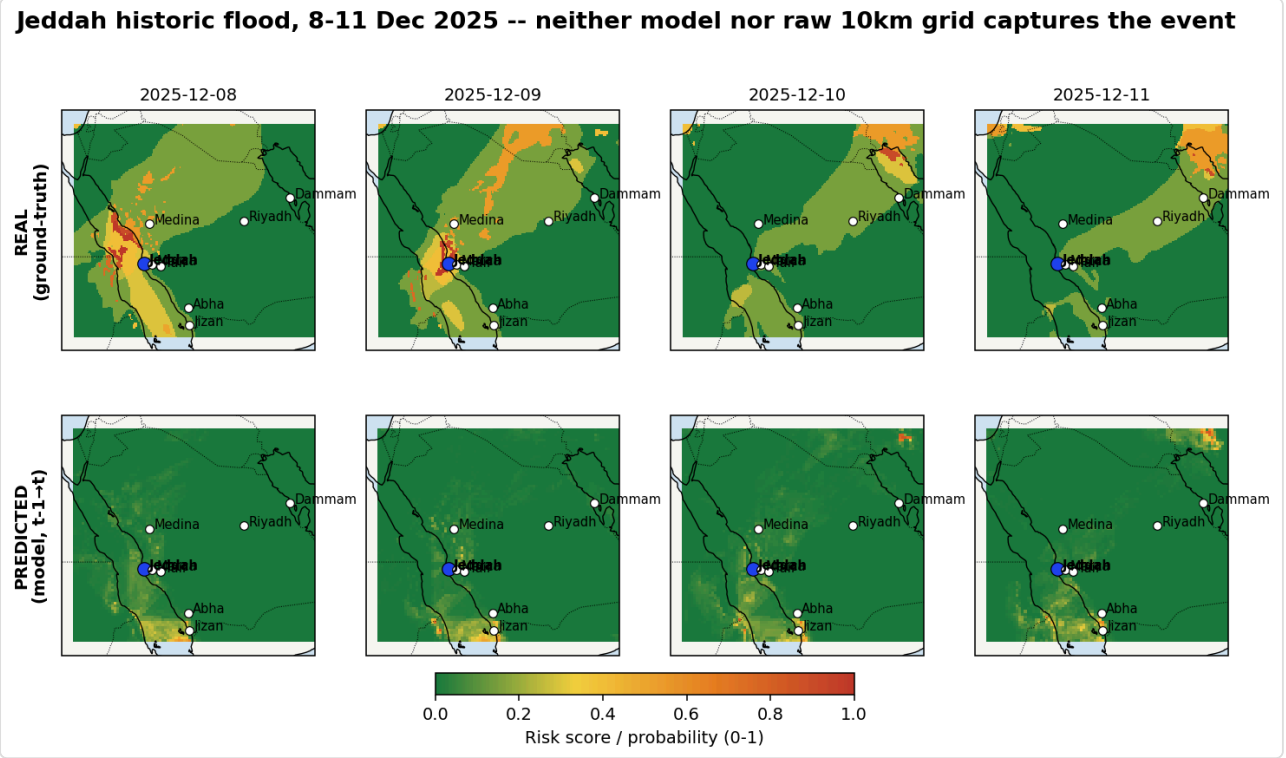


Dust storm (Dammam): the REAL row shows an orange/red band tracking along the Gulf coast near Dammam on 17-18 May, consistent with the city-level detection already reported (§9.13-adjacent). The PREDICTED row shows a broader, noisier high-risk area — an honest difference, not a bug: the model's probability surface is continuous and less spatially sharp than the rule engine's threshold-based field.



Heatwave (Dammam): this map makes a real limitation visible that a single-city time series did not: the REAL row stays green/yellow at Dammam across the entire window (the rule engine requires tmax≥45°C locally, and Dammam's grid cell peaked at 44.2°C), while the PREDICTED row shows sustained high risk. The model's call was directionally correct and matched the reported regional heat

event (whose 49.3°C record was measured at Al-Ahsa, not one of this system's 8 modeled cities), but the rule-engine ground-truth check does not independently confirm it at Dammam's exact coordinates the way it does for the dust storm. This inconsistency between how strictly the two events were checked was caught while building this map and is disclosed here rather than smoothed over.



Flash flood (Jeddah): both rows stay green at Jeddah itself across all 4 days, consistent with §11's disclosed resolution limitation. One caveat worth stating plainly: the REAL row does show a red cell on 9 December, but it sits roughly 165 km north of Jeddah (near the coast, not at any of the 8 modeled cities) and reflects a separate, genuine rain signal in the raw data (18 mm precipitation, CAPE 1700, IVT 339 — all real values at that cell) — not the Jeddah event itself. It is included unedited because removing it would understate what the raw grid actually contains; a careless read of the map could otherwise mistake it for confirmation at Jeddah, which it is not.

Animated day-by-day GIF versions of all 3 maps were also generated for cases where the sequence is easier to read as motion than as still panels; they are provided as separate files alongside this report and are not yet part of the deployed GitHub Pages site.

12.1 Extended verification: 9 further events from the full reviewer-provided calendar

The 3 events mapped above are the detailed, spatially-visualized subset of a larger, full 2025 event calendar reviewers provided (4 heatwave waves, 3 dust storms, and — per the source text's own count, though it only actually lists 5 date windows — "6" flood events). All remaining 9 date windows were verified using the same method (`DetectionEngine.risk_field` independent rule score vs. `forecast_tool` model probability, at the specific named city where available). Full per-event detail, including two further honest corrections found during this pass, is in the companion report

`MAZU_Real_Hadise_Dogrulama_EN.pdf` (Chinese: `..._CN.pdf`); the table below summarizes all 12 events verified in total.

Event	Verdict
Dammam dust storm (17 May)	✅ HIT
Dammam heatwave (13 June)	⚠️ Partial hit (corrected)
Jeddah historic flood (9-10 Dec)	❌ MISS — data-resolution limitation
Haboob dust storm (4-5 May)	— Inconclusive, outside coverage
Dust storm (30 Jun-5 Jul)	✅ Strong HIT
Heatwave, 2nd wave (28 Jun-5 Jul)	❌ MISS
Heatwave, 3rd wave (20-27 Jul)	⚠️ Partial hit, with clarification
Heatwave, 4th wave (29 Jul-5 Aug)	⚠️ Partial hit
Flood (6-7 Jan)	— Inconclusive, data gap + within training period
Flood (6-7 Mar)	❌ MISS / no signal
Flood (14 Aug, Taif)	⚠️ Model/real disagreement
Flood (27-28 Aug)	⚠️ Mixed result

Across all 12: 3 clean hits, 3 clean misses, 4 partial/mixed, 2 inconclusive for stated methodological reasons — an uneven, honest distribution, not a curated "everything works" sample. One of the 9 additional checks also caught a second instance of the same verification-rigor gap already disclosed above (§12): reviewers' 20-27 July alert named Dammam and Riyadh at 45-50°C, but both cities' own raw grid temperatures stayed below 44.3°C throughout — while an independent, genuine hit was found instead at Mecca (22 July). See the companion report for the full reasoning behind every verdict.

13. Live Deployment

The full system, including all reports and this project's public-facing documentation, is deployed at `https://turgutino.github.io/mazu-saudi-warning/` (GitHub: `turgutino/mazu-saudi-warning`). Every extension above was verified live on the deployed site, not just locally, before being reported complete.

14. What We're Considering Next

Idea	Est. effort	Status

Arabic-language agent responses	Low–medium	Under consideration
Formal KG-selection comparison document (why our schema vs. SWEET/GeoSPARQL/WMO Ontology/KnowWhereGraph/etc.)	Low	Under consideration
Drought as a 4th hazard (full depth: dataset, detection, model, KG)	High	Under consideration
KnowWhereGraph external linkage	Medium	Deprioritized — marginal expected value vs. our already-complete KG for the 8 covered cities/3 hazards
Updated slide deck reflecting current state	Medium	In progress
Human-review workflow to promote literature_evidence_tool candidates into the formal, verified KG	Low–medium	Under consideration
In-process multi-hop evidence graph for literature_evidence_tool (networkx, no external DB) — chain facts across documents instead of single-document top-k (§9.13)	Low–medium	Deprioritized for now — corpus too small (12 entries) for the expected gain to justify the work
Multi-day forecast horizon (2-day, 3-day, weekly outlooks beyond the current 1-day-ahead) (§6)	Medium–High	Under consideration — raised directly by advisor/reviewer feedback
Finer spatial resolution (sub-10km) to better capture short-duration, highly localized convective events like the missed 9-10 Dec 2025 Jeddah flood (§11)	High	Under consideration — depends on availability of finer-resolution source data

Appendix — Full Statistics Snapshot

Metric	Value
Hazards covered	3 (flash flood, heatwave, dust storm)
Cities covered	8 (Jeddah, Mecca, Riyadh, Jizan, Dammam, Taif, Medina, Abha)
KG nodes / edges	60 / 183
Peer-reviewed citations grounding mechanisms	6 (from 7 cited publications)

Agent tools	7
Unit tests (agent/02_test_tools.py + calibration/ensemble test scripts)	237 (169 + 68), 0 failing
Full-system audit checks (FULL_SYSTEM_AUDIT.py)	121 / 121 passed
Raw source data	365 daily NetCDF files, ~5 GB, 0.1° resolution
Train / test split	Jan–Jun 2025 train, Jul–Dec 2025 test (time-based, no leakage)
Live site	turgutino.github.io/mazu-saudi-warning